

QualiJournal 관리자 시스템 이슈 분석 및 해결 보고서

목차

1. [문제 개요 및 증상 정리](#)
2. [원인 분석](#)
3. [전체 점검 체크리스트](#)
4. [Cloud Run 리비전 및 트래픽 업데이트](#)
5. [인증 토큰 적용 및 Fetch 재정의](#)
6. [UI 코드 수정 사항](#)
7. [PowerShell 배포 스크립트 주의사항](#)
8. [QA 자동 점검 버튼 동작 확인 및 개선](#)
9. [도메인 캐시 무효화 방법](#)
10. [Definition of Done 체크리스트](#)

문제 개요 및 증상 정리

QualiJournal 관리자(admin) 시스템에서 다음과 같은 여러 문제가 **동시에** 보고되었습니다:

- **UI 상 버튼 동작 불가:** “QA 자동 점검” 버튼을 비롯한 몇몇 UI 버튼 클릭에 반응이 없습니다.
- **KPI 수치 0 고정:** 주요 KPI 지표가 0에서 갱신되지 않으며, 정상 데이터가 표시되지 않습니다.
- **토큰 입력 실패:** 관리 도구 상단의 **API 토큰 입력** 기능이 동작하지 않아, 토큰을 입력해도 인증이 적용되지 않습니다.
- **보고서/CSV 내보내기 결과 없음:** “보고서 생성”이나 “CSV 내보내기” 기능을 실행해도, **응답이 없거나 빈 결과**가 반환됩니다.
- **배포 후 신규 UI 불일치:** 최신 프론트엔드(`index.html`)를 배포했음에도 **이전 UI 화면**이 계속 노출되어, 수정된 UI가 적용되지 않습니다.
- **Cloud Run 배포 스크립트 오류:** 배포 스크립트(PowerShell) 실행 중 `gcloud run services update-traffic` 명령어에서 **오류 발생**으로 **최신 리비전으로 트래픽 전환 실패**. Cloud Run 로그 상 새 리비전이 생성되었으나, 트래픽이 이전 리비전에 남아있는 상황입니다.
- **API 호출 오류 상황:**
 - `/api/status` 요청 시 **HTTP 401 오류** 또는 **응답 값 0만** 반환됨 (인증 실패 또는 데이터 없음).
 - `/api/report` 요청은 **HTTP 200 OK**가 나오나, **실제 데이터는 비어있음** (빈 결과).
 - `/api/auth/mode` 요청 시 **HTTP 404 오류** 발생 (엔드포인트를 찾을 수 없음).
- 위 증상들로 미루어, **인증 토큰 헤더 미첨부** 또는 **백엔드 인증 로직 문제**가 의심됩니다.

이러한 문제가 동시다발적으로 발생하여 관리자 도메인 (`admin.standardai.co.kr`) 상에서 서비스 기능이 거의 마비된 상태입니다. 아래에서는 각 증상의 근본 원인을 분석하고, 구조적인 해결 방법을 제시합니다.

원인 분석

위 증상들을 종합한 결과, 문제의 주요 원인은 **프론트엔드(UI)**, **백엔드(Cloud Run)**, **인증 토큰 처리**, **배포 절차(PowerShell 스크립트)**, **캐시/리비전 관리**의 다섯 가지 측면에서 나타났습니다:

- **UI 이벤트 바인딩 누락:** 새로 추가되거나 변경된 UI 요소(예: QA 자동 점검 버튼)에 대한 **이벤트 핸들러 연결이 누락**되었습니다. 그 결과 버튼을 클릭해도 아무 동작을 하지 않았습니다. 예를 들어, `index.html`에 해당 버

튼의 `onclick` 스크립트나 이벤트 리스너가 정의되어 있지 않은 것으로 보입니다. 또한, 프론트엔드 자바스크립트 코드에서 `fetch` 함수 재정의 오류가 발생하여, 초기화 단계에서 **무한 재귀**나 예외가 일어나면서 **아래 코드 실행이 중단**되고 다른 UI 바인딩이 모두 무효화되었을 가능성도 있습니다. (이후 분석에서 `fetch` 재정의 관련 코드를 다룹니다.)

- **Cloud Run 리비전 미반영 및 캐시:** 최신 프론트엔드 (`index.html`)가 배포되었음에도 **사용자에게 구버전 UI가 표시된** 원인은 **Cloud Run 최신 리비전에 트래픽이 할당되지 않았거나, 캐시 문제로** 판단됩니다. 일반적으로 Cloud Run은 새 revision 배포 시 자동으로 **100% 트래픽을 최신 revision에 할당**합니다 ¹. 그런데 이번 배포 과정에서 `gcloud run services update-traffic --to-latest` 명령 오류로 인해 트래픽 전환이 이루어지지 않아, 이전 리비전의 UI가 계속 서비스된 것으로 추정됩니다. 한편, 사용자 측 **브라우저 캐시**나 **도메인 CDN 캐시**도 원인일 수 있습니다. 실제 사례에서도 **브라우저/ CDN 캐시, 로드 밸런서, DNS 캐시** 등이 갱신을 방해해 **구버전이 표시되는 현상**이 보고됩니다 ². 이번 경우 Cloud Run 리비전 미전환과 결합되어, 캐시에 구버전 UI가 남게 된 것으로 보입니다.

- **인증 토큰 부착 로직 오류:** 관리 도구가 API 호출 시 **Authorization 토큰**을 제대로 **헤더에 첨부하지 못한 문제**가 발견되었습니다. 프론트엔드에서 API 호출 (`/api/status`, `/api/report` 등)을 수행할 때 **HTTP Authorization 헤더에 Bearer 토큰**을 포함시켜야 하는데, 해당 로직이 누락되거나 잘못 구현되었습니다. 이로 인해 백엔드가 **모든 요청을 인증되지 않은 것으로 간주**하여 `/api/status`에 401 또는 기본값(0)을 응답하고, `/api/report`도 **데이터 없이** 응답한 것입니다. 특히, `window.fetch` 재정의 코드가 잘못되어 **Authorization 헤더 설정에 실패**했을 가능성이 큼니다. 잘못된 재정의로 인해 `window.fetch`가 자기 자신을 재귀 호출하여 **Maximum call stack exceeded** 오류(스택 오버플로우)가 발생했을 수도 있습니다. 실제로 한 개발 사례에서 이러한 코드가 **Chrome에서 스택 오버플로우를 일으켰으며**, `window.fetch`를 재정의할 때 **원본 fetch 함수를 가리키는 별도 변수 (`origFetch`)를 사용하여 호출**함으로써 문제를 해결할 수 있음이 보고되었습니다 ³. (즉, `fetch` 재정의시 원본 함수를 사용하지 않고 `window.fetch`를 다시 호출하면 자기 자신을 부르는 셈이 되어 **무한 재귀**가 발생합니다.)

- **Orchestrator 백엔드 경로 및 인증 문제:** 백엔드 Cloud Run 서비스 (`quali-admin-domap`)에서 **인증 모드 및 경로 설정**에 일부 문제점이 확인되었습니다. `/api/auth/mode` 엔드포인트가 404를 반환한 것은, 해당 경로를 백엔드 앱(`orchestrator.py`)에 **구현하지 않았거나 라우팅 설정 누락**으로 보입니다. 원래 의도는 **현재 인증 방식을 조회**하여 프론트엔드가 **로그인 모드인지 토큰 모드인지 결정**하려던 것으로 추측됩니다. 그러나 엔드포인트 부재로 404가 발생했고, 프론트엔드 쪽에서 이 예외를 처리하지 않아 **토큰 입력 UI 표시 등 후속 동작이 진행되지 않았을** 가능성이 있습니다. 또한 `orchestrator.py` 수정 사항이 배포되었으나, **위의 리비전 트래픽 미전환 이슈로 실제 적용되지 않았을** 가능성이 있습니다. 그 결과 백엔드 측에서 QA 버튼 동작이나 토큰 인증 관련 수정 효과가 나타나지 않아, **QA 자동 점검 API 호출이 계속 실패**했을 수 있습니다. 한편, `/api/status`가 토큰 미첨부 시 **401 Unauthorized**가 나온 것은 백엔드에서 **인증 데코레이터나 미들웨어**가 적용되어 토큰 없는 요청을 거부한 것으로 보이며, 이는 올바른 동작이나 프론트의 토큰 전송 누락 때문에 발생한 것입니다. `/api/report`의 200 응답/빈 결과는, 아마도 토큰 없이 호출된 경우 **디폴트 빈 데이터**를 돌려주도록 구현되어 있기 때문일 수 있습니다 (이 부분은 구현에 따라 다를 수 있습니다). **Orchestrator 실행 실패**라는 표현은, 아마 **QA 자동 점검 기능(API)**이 내부적으로 다른 서비스 호출이나 비동기 작업을 하는데, 토큰 누락 또는 기타 이유로 **실행이 아예 트리거되지 못한 상태**를 가리킬 것입니다.

- **배포 스크립트(PowerShell) 오류:** CI/CD 또는 수동 배포에 사용된 PowerShell 스크립트에서 **gcloud CLI 명령 구문 오류**가 있었습니다. 특히 `gcloud run services update-traffic` 명령어 실행 시 **따옴표 처리 문제**로 인한 오류가 보고되었습니다. Windows PowerShell에서는 외부 CLI 호출 시 인자에 대한 따옴표 처리에 유의해야 합니다. 잘못된 따옴표 사용은 인자가 전달되지 않거나 잘못 해석되는 원인이 됩니다. 실제로 PowerShell에서는 **명령행 인자의 이중인용부호를 적절히 escape하지 않으면 제거(strip)해버리는** 동작이 확인된 바 있습니다 ⁴. 예컨대 PowerShell에서 `someProgram "arg"` 형태로 호출하면, 이 **따옴표가 프로그램에 그대로 전달되지 않을 수 있으며**, 복잡한 인용부호 조합은 별도로 백슬래시 등을 이용해 이스케이프해야 합니다 ⁵. 이번 스크립트에서도 인자를 감싸는 따옴표 사용이나 변수 치환 부분에서 오류가 있어, `--to-`

`latest` 플래그나 서비스 이름/리전 인자가 제대로 전달되지 않았을 것으로 보입니다. 그 결과 Cloud Run의 트래픽 라우팅 업데이트 명령이 실패했고, 새 리비전이 생성되고도 **트래픽이 0% (할당 안 됨)**으로 남게 되어 UI 업데이트가 반영되지 않았습니다.

요약하면, **프론트엔드의 이벤트/토큰 처리 문제, 백엔드의 인증 엔드포인트 및 배포 리비전 문제, 배포 자동화 스크립트 오류**, 그리고 **캐시/리비전 전환 이슈**가 복합적으로 작용하여 이러한 증상들이 나타났습니다.

전체 점검 체크리스트

문제 해결을 위해 **프론트엔드부터 백엔드, 배포 환경 전반에 걸친 종합 점검**이 필요합니다. 다음은 주요 점검 항목들의 체크리스트입니다:

• 프론트엔드(UI) 점검:

- [] `index.html` 및 관련 JS 코드에 **필요한 이벤트 바인딩**이 모두 구현되어 있는지 확인 (특히 새로운 버튼들의 `onclick` 또는 `addEventListener` 존재 여부).
- [] **토큰 입력 필드 및 버튼**이 제대로 동작하는지 확인 (토큰을 저장하고 이후 요청에 적용하는 로직 확인).
- [] `window.fetch` **재정의 코드** 검토: 재정의가 올바르게 구현되어 Authorization 헤더 추가와 **원본 fetch 호출**이 정상적으로 이뤄지는지 확인. (재정의 시 **무한 재귀 발생 여부** 확인).
- [] 프론트엔드 콘솔에 **JavaScript 오류**가 없는지 확인 (개발자 도구 열어서 Error 로그 확인).

• 백엔드(Orchestrator 및 API) 점검:

- [] Cloud Run에 배포된 `orchestrator.py` **최신 버전**이 이번 문제 해결을 위한 수정사항(예: 토큰 처리 등)을 포함하고 있는지 확인.
- [] `/api/auth/mode` **엔드포인트 구현 여부** 확인: 없을 경우 프론트엔드에서의 사용 방식 변경 or 백엔드에 해당 경로 추가를 검토.
- [] **인증 실패 시 백엔드 응답** 확인: `/api/status`가 인증 없을 때 401이 나오는지, `/api/report`는 어떤 동작을 하는지 명확히 파악하여 프론트엔드 처리와 맞추기.
- [] Orchestrator 로그 점검: QA 버튼 동작 시 백엔드 로그에 **요청 수신/처리 로그**가 남는지 확인하고, **예외 발생 여부** 체크. (토큰 유효하지 않거나 내부 API 호출 실패 로그 등)

• Cloud Run 서비스 설정 점검:

- [] **최신 리비전 배포 및 트래픽 할당** 확인: Cloud Run 콘솔 또는 CLI로 서비스 `quali-admin-domap`의 현재 **트래픽 분배 상태**를 확인 (정상이라면 **latest 리비전에 100%**여야 함).
- [] **리전 및 도메인 설정** 확인: `admin.standardai.co.kr` 커스텀 도메인이 해당 서비스의 올바른 리비전을 가리키고 있는지 (Cloud Run 도메인 매핑 상태, SSL 등도 함께 점검).
- [] **환경 변수/설정** 확인: 혹시 백엔드 서비스가 필요로 하는 API 토큰 등을 **ENV로 주입**받는 구조라면, 해당 환경변수가 제대로 설정돼있는지 (예: 내부 시스템 호출을 위한 키 등이 필요할 수 있음).

• 캐시 및 배포 버전 점검:

- [] **브라우저 캐시**: 최종 사용자 브라우저에서 **강력 새로고침(CTRL+F5)** 시 새로운 UI가 보이는지 확인. 또는 **개발자 도구 > Network 탭**에서 “**Disable cache**” 옵션으로 새 요청 테스트.
- [] **CDN/프록시 캐시**: 만약 Cloud Run 커스텀 도메인에 **Cloud CDN**을 사용 중이거나 CloudFlare 같은 CDN을 앞단에 썼다면, **캐시 무효화(Purge)**를 수행. (Cloud Run 자체는 **동적 콘텐츠는 기본 비캐시**이지만 ², 정적 파일이 CDN에 캐싱될 수 있으므로 확인 필요)

- [] **DNS 캐시**: 도메인의 CNAME/A 레코드가 변경된 경우 (이번엔 아니겠지만) 전파 여부 확인.

• 배포 스크립트 및 CI 파이프라인 점검:

- [] PowerShell 스크립트의 **gcloud 명령어 구문** 재검토: 특히 `gcloud run deploy` 및 `gcloud run services update-traffic` 부분의 **인자 전달**이 올바른지 확인. (예: `$serviceName` 변수 사용 시 인용 부호 문제, `--region` 인자 누락 여부 등)
- [] **로그 확인**: 스크립트 실행 로그에서 **에러 메시지** 재확인. (PowerShell에서 인자 관련 에러는 따옴표나 공백 처리 문제를 암시함)
- [] **수정 후 재배포 테스트**: 스크립트 수정 후 Cloud Run에 **정상 배포 및 트래픽 전환**이 한 사이클로 이뤄지는지 (새 revision 생성 -> 즉시 100% 할당 -> 서비스 응답 확인) 검증.

위 항목들을 차례로 확인하고 하나씩 해결해 나갔습니다. 다음 섹션에서는 주요 이슈들에 대한 **구체적인 수정 방법과 재발 방지 대책**을 정리합니다.

Cloud Run 리비전 및 트래픽 업데이트

앞서 확인되었듯, **최신 UI가 반영되지 않은** 주된 원인은 **Cloud Run의 새 리비전에 트래픽이 안 올라간** 것이었습니다. 정상적인 Cloud Run 배포의 경우, 새 **revision**이 생성되면 **자동으로 100% 트래픽이 최신 revision으로 할당**됩니다 ¹. 그러나 수동 설정 또는 이전에 **트래픽을 수동 분할/고정(태그링)**한 적이 있으면 “manual” 모드가 되어 자동 전환이 안 될 수 있습니다. 이때는 별도로 명령을 실행해줘야 합니다.

해결: 문제 해결을 위해 **최신 revision으로 트래픽을 전환하는 명령**을 올바르게 실행했습니다. 구체적인 명령어는 다음과 같습니다:

```
# Cloud Run 서비스의 모든 트래픽을 최신 리비전에 할당 (asia-northeast3 리전 예시)
gcloud run services update-traffic quali-admin-domap --to-latest --region=asia-northeast3
```

위 명령은 `quali-admin-domap` 서비스의 트래픽을 강제로 **가장 최근에 배포된 리비전(latest)**으로 100% 돌리는 역할을 합니다 ⁶. 실행 후 Cloud Run 콘솔의 해당 서비스 “**트래픽 분할**” 설정을 확인하면, 최신 리비전 옆에 “100%”로 표시되고 이전 리비전들은 0%로 바뀝니다. (만약 수동으로 이전에 **tag**를 지정하여 특정 리비전에 고정했다면, `--to-latest`로 자동 모드로 전환해야 이후 deploy 시 자동 갱신이 이루어집니다.)

PowerShell 관련 주의: 이번에 PowerShell 스크립트에서 `--to-latest` 옵션 전달에 실수가 있었던 만큼, **Windows 환경에서 gcloud 명령을 스크립팅할 때의 인용부호 사용**을 점검했습니다. PowerShell에서는 **인자를 문자열로 감싸면** 다르게 해석되거나, 경우에 따라 **따옴표가 제거되어** 전달될 수 있습니다 ⁴. 따라서 **굳이 인자를 감싸는 따옴표를 쓰지 않고**, 위 예시처럼 **플래그와 값을 명확히 분리**하거나 `--flag=value` 형식으로 전달하는 것이 좋습니다. 예를 들어, 서비스 이름이나 region처럼 **공백 없는 값**은 따옴표 없이 그대로 쓰고, 만약 값에 공백이 필요하다면 `"`로 감싸되 PowerShell 문자열 규칙에 맞게 사용합니다.

예시:

```
# 올바른 예 - 따옴표 최소화
$service = "quali-admin-domap"
gcloud run services update-traffic $service --to-latest --region asia-northeast3
```

위와 같이 작성하면 PowerShell이 인자를 잘 나누어 전달합니다. 반면,

```
gcloud run services update-traffic "quali-admin-domap" "--to-latest"
```

처럼 모든 인자를 큰따옴표로 감싸면 `--to-latest` 가 문자열로 인식되어 **의도와 다르게 파싱될 수** 있습니다.

일반적으로 **gcloud CLI 스크립팅 시 PowerShell**에서는 백틱

`( )** 문자를 줄 끝에 붙여 여러 줄로 나누거나, $Env:VAR` 등을 활용해 환경변수를 설정하는 방법도 있습니다. 중요한 것은 인자의 따옴표를 최소화하고, 필요한 경우 백슬래시로 이스케이프**하여 PowerShell이 외부 프로그램에 정확히 인자를 넘기도록 하는 것입니다 5 .`

Cloud Run 최신 리비전으로 트래픽을 넘긴 후, **곧바로 새로운 UI가 표시되기** 시작했습니다. 브라우저 캐시 때문에 바로 보이지 않을 수 있어, **Shift+새로고침** 등을 통해 강제 갱신하니 최신 UI가 확인되었습니다. 만약 일부 사용자에게 여전히 구버전이 보인다면, Cloud Run 도메인에 **Cache-Control** 헤더를 설정하여 배포하거나(예: `no-cache, no-store`), Cloudflare 같은 경우 **캐시 퍼지**를 수행하면 해결됩니다. Stack Overflow의 사례에 따르면, Firebase Hosting 같은 CDN 사용 시에도 `Cache-Control: no-cache` 헤더로 **배포 즉시 새로운 콘텐츠** 서빙을 유도할 수 있습니다 ⁷ . 이번 서비스는 Cloud Run 커스텀 도메인을 직접 사용하므로, 기본적으로 **동적 응답은 CDN 캐시가 없지만**(Cloud Run 도메인 매핑 + Cloud CDN을 사용하지 않은 경우), **정적 리소스(예: JS/CSS)**가 브라우저에 캐시되었을 수 있으므로, 필요시 HTML에 `<meta http-equiv="Cache-Control" content="no-cache">` 등을 추가 검토했습니다.

정리하면, **Cloud Run 측면의 수정은 (1) 최신 리비전에 트래픽 할당 및 (2) 캐시 무효화로** 요약되며, 이로써 **배포된 최신 코드가 사용자에게 적용되도록** 했습니다.

인증 토큰 적용 및 Fetch 재정의

다음으로, **인증 토큰 처리 문제**를 해결했습니다. 관리자 시스템은 API 호출 시 **사전에 발급받은 토큰**을 Authorization 헤더에 포함하여 백엔드에 전달해야 정상 응답을 받을 수 있습니다. 그러나 이전 버전에서는 이 과정이 누락되어 모든 API 요청이 **인증 없음(Unauthorized)** 상태였습니다.

해결: 프론트엔드에서 **API 호출 전반에 토큰을 자동 부착**하도록 코드를 수정했습니다. 두 가지 방안을 고려했는데, **간단한 Fetch 인터셉트(override) 적용** 방법을 선택했습니다:

1. **window.fetch 인터셉트 사용:** 앱이 로드될 때 `window.fetch` 를 오버라이드하여, 호출되는 모든 fetch 요청에 **자동으로 Authorization 헤더를 첨부**하도록 구현했습니다. 구체적으로는, **원본 fetch 함수를 저장**해 두고, 새 fetch 함수에서 **헤더를 추가한 뒤 원본 함수를 호출**합니다. 코드 예시는 다음과 같습니다:

```
<script>
// ① 기존 fetch 함수를 저장
const originalFetch = window.fetch;
// ② window.fetch 재정의: 토큰 자동 첨부
window.fetch = async function(resource, options = {}) {
  const token = localStorage.getItem('apiToken'); // 토큰 저장 위치
  if (token) {
    // Authorization 헤더 추가 (기존 헤더 보존)
    options.headers = {
      ...options.headers,
      'Authorization': `Bearer ${token}`
    };
  }
}
```

```
// ③ 원본 fetch 함수 호출하여 실제 요청 수행
return originalFetch.apply(this, [resource, options]);
};
</script>
```

위 코드가 적용되면, 이후 실행되는 `fetch('/api/....')` 호출에는 자동으로 `Authorization: Bearer ...` 헤더가 붙게 됩니다. 중요 포인트는 **재정의 시 원본** `originalFetch` 를 호출하는 것인데, 이는 **무한 재귀를 방지**하기 위함입니다. 앞서 문제 원인 분석에서 추측한 대로, 이전 코드에서는 잘못된 재정의로 `window.fetch` 가 내부에서 다시 `window.fetch` 를 호출하는 실수를 했을 가능성이 큼니다. 이는 **콜 스택 초과**를 유발하므로, 반드시 위 예시처럼 **별도 변수에 저장한 원본 fetch**를 써야 합니다. (실제로 한 JavaScript 개발 답변에서도, Chrome에서 `fetch` 재정의시 스택이 팍 차는 문제가 지적되어 `const origFetch = window.fetch; ... origFetch(...args)` 형태로 수정하면 문제가 해결된다고 조언합니다 ³.)

1. API 호출부 개별 수정: fetch 인터셉트를 쓰지 않는다면, 각 API 호출 코드마다

`fetch(url, { headers: {Authorization: 'Bearer <token>'} })` 식으로 헤더를 넣어주는 방법도 있습니다 ⁸. 그러나 이 방법은 기존 코드 여러 곳을 수정해야 하고 **중복**이 생기므로, 유지보수를 고려해 **중앙 집중식 방법(인터셉트)**를 택했습니다.

위 수정에 따라, **토큰 저장 및 활용 방식**도 점검했습니다. 사용자가 토큰을 입력하면, 해당 값을 `localStorage`나 전역 변수에 저장해두고 이후 `fetch` 재정의에서 참조합니다. 예컨대 `id="token-input"` 필드와 “등록” 버튼이 있다면 다음과 같이 구현했습니다:

```
document.getElementById('setTokenBtn').addEventListener('click', () => {
  const tokenVal = document.getElementById('token-input').value.trim();
  if (tokenVal) {
    localStorage.setItem('apiToken', tokenVal);
    alert('API 토큰이 설정되었습니다.');
```

// 토큰 설정 후 즉시 KPI 현황 갱신 호출

```
    fetch('/api/status')
      .then(res => res.json())
      .then(data => updateKpiDisplay(data))
      .catch(err => console.error('Status fetch error:', err));
  } else {
    alert('토큰 값을 입력해주세요.');
```

이렇게 하면 관리자가 토큰을 입력->저장하는 즉시 `/api/status`를 다시 불러와 KPI 값을 갱신하도록 할 수 있습니다. (이전에는 토큰을 안 보내 401로 받았지만, 이제는 Authorization 헤더 포함으로 200과 실제 값을 받을 것입니다.)

Authorization 헤더 형식: 한 가지 유의할 점은 **토큰 앞에 “Bearer “ 문자열을 붙이는 것**입니다. 백엔드에서 **JWT** 또는 **OAuth2 Bearer** 토큰 방식을 사용하는 경우, 헤더값에 `Bearer` prefix가 필요합니다. (`orchestrator.py`에서 해당 헤더를 파싱하여 인증을 확인할 가능성이 높음) 이 부분도 누락되지 않도록 확인했습니다. (일반적인 Bearer 토큰 사용 예시는 MDN이나 ReqBin 등에서도 확인할 수 있습니다 ⁸.)

백엔드 인증 모드 대응: `/api/auth/mode`가 404인 상황에 대비하여, 프론트엔드 코드에서 이 호출 부분을 **예외** 처리했습니다. 즉, **404가 나오면 기본적으로 “토큰 인증 모드”로 간주**하도록 하였습니다. 기존 코드가 아마 다음과 같았을 것입니다:

```

fetch('/api/auth/mode')
  .then(res => res.json())
  .then(data => {
    if (data.mode === 'login') {
      showLoginForm();
    } else if (data.mode === 'token') {
      showTokenInput();
    }
  })
  .catch(err => {
    console.error('/api/auth/mode error:', err);
    // 수정: 에러 발생 시 기본적으로 토큰 인증 방식으로 진행
    showTokenInput();
  });

```

이렇게 수정하여, `/api/auth/mode`가 없더라도 **토큰 입력 UI를 항상 노출**시킵니다. 실제 운영 상 관리자 시스템은 사전 발급된 API 토큰을 입력해 사용하는 구조로 파악되므로, 굳이 로그인 모드를 고려하지 않고 바로 토큰 입력을 받게 해도 무방할 것입니다. (추후 백엔드에 해당 엔드포인트를 구현해주면 보다 깔끔하겠지만, 프론트엔드 단의 워크어라운드로도 문제는 해결됩니다.)

이상의 변경을 통해, 이제 **API 호출 시 올바른 토큰이 헤더에 실려 전송되며, 백엔드에서 401 오류 없이 실제 데이터 (KPI, 보고서 등)를 응답**하게 되었습니다. 수동 테스트 결과 `/api/status` 응답이 0이 아닌 실제 KPI 값으로 채워졌고, `/api/report` 호출 시도 시에도 응답 데이터가 생성되는 것을 확인했습니다.

UI 코드 수정 사항

UI 측면에서는 **이벤트 핸들러 연결**과 **오류 처리 보완** 작업을 진행했습니다. 특히 **QA 자동 점검 버튼** 관련 코드를 점검한 결과, 다음과 같은 수정/구현이 이루어졌습니다:

- **QA 버튼 이벤트 바인딩:** 해당 버튼 (`id="btn-run-qa"` 등) 요소를 찾아 클릭 시 호출될 함수를 연결했습니다. 예를 들어:

```

document.getElementById('btn-run-qa').addEventListener('click', () => {
  runQAAutomation();
});

```

그리고 `runQAAutomation()` 함수 내에서는 `/api/qa/run` 또는 유사한 API를 호출하도록 구현했습니다. (실제 경로는 시스템 구현에 따라 다를 것이므로 로그나 코드를 참고하여 설정) 이전에는 이 부분이 아예 없어서 버튼이 눌러져도 아무 일이 일어나지 않았습니다.

- **QA 실행 API 호출 구현:** 백엔드에서 QA 자동 점검을 트리거하는 API가 있다면, 해당 경로로 fetch 요청을 보내도록 했습니다. 또한 이 요청에도 **토큰 헤더가 포함**되어야 하므로, 앞서 설정한 fetch 인터셉트가 적용됩니다.

```

function runQAAutomation() {
  // UI 피드백: 로딩 중 표시
  showSpinner('QA 검사 실행 중...');
  fetch('/api/qa/run', { method: 'POST' })

```

```

.then(res => res.json())
.then(result => {
  hideSpinner();
  if (result.success) {
    alert('QA 자동 점검이 시작되었습니다. 잠시 후 결과를 확인하세요.');
```

// 필요하면 결과 표시 기능 호출 등

```

  } else {
    alert('QA 점검 실행에 실패했습니다: ' + (result.message || 'Unknown error'));
  }
})
.catch(err => {
  hideSpinner();
  console.error('QA run error:', err);
  alert('QA 점검 요청 중 오류 발생!');
});
}

```

위 코드는 예시이며, 실제 API 경로와 응답 프로토콜에 맞춰 구현했습니다. 중요한 것은 **성공/실패에 따른 사용자 피드백**을 추가한 점입니다. 이전에는 버튼 눌러도 반응이 없어서 사용자가 인지하지 못했는데, 이제 **alert**이나 화면상 메시지로 “**실행됨/오류**” 여부를 알 수 있습니다.

- **CSV/보고서 내보내기 버튼 동작 확인:** 이 버튼들도 유사하게 이벤트가 연결돼있는지 확인하고, 필요시 `fetch('/api/report', ...)` 호출 후 **파일 다운로드**를 처리하는 코드를 개선했습니다. 예컨대:

```

document.getElementById('btn-export-csv').onclick = () => {
  fetch('/api/report')
    .then(res => res.blob())
    .then(blob => {
      const url = URL.createObjectURL(blob);
      const a = document.createElement('a');
      a.href = url;
      a.download = 'report.csv';
      document.body.appendChild(a);
      a.click();
      a.remove();
    })
    .catch(err => {
      console.error('Report export failed:', err);
      alert('CSV 내보내기에 실패했습니다.');
```

이렇게 하면 `/api/report`의 응답 스트림을 CSV 파일로 다운로드 처리합니다. (백엔드가 CSV를 만들어주고 있었다면 이로 충분하고, JSON을 내려주는 거라면 프론트에서 CSV 변환이 추가로 필요할 수 있습니다.)

- **전역 오류 핸들러/경고:** 혹시 UI 코드 내 **예외가 발생**해 전체 스크립트 실행이 중단되는 것을 완화하기 위해, `window.onerror`를 활용해 오류를 캐치하고 로그를 수집하는 것도 고려했습니다. 예를 들어:


```

window.onerror = function(message, source, lineno, colno, error) {
  console.error("Global Error Caught:", message, "at", source, lineno+":"+colno);
  // 필요시 서버로 로그 전송 또는 사용자에게 안내
};

```

이렇게 하면 fetch 재정의 같은 부분에서 실수가 있어도 콘솔에 바로 기록되고 문제를 빠르게 발견할 수 있습니다.

- **UI 표시 내용 업데이트:** KPI 수치를 표시하는 DOM 요소가 0으로 남아있던 부분은, 토큰 설정 후 **정상 데이터 수신 시 해당 요소의 innerText를 업데이트**하도록 했습니다. 또한 데이터가 없을 경우에도 0으로만 두지 않고 “(데이터 없음)” 등으로 표시하거나, **토큰 미입력 시 “토큰을 입력해주세요” 안내 메시지를 KPI 영역에 보여주는 개선도 함께 이루어졌습니다.**

위 수정 사항 적용 후, **UI 레벨에서의 모든 버튼과 기능이 의도대로 동작하는 것**을 확인했습니다. QA 자동 점검 버튼을 누르면 백엔드 호출이 이뤄지고 (개발자 도구 Network 탭에서 202 Accepted 등 응답 확인), KPI 새로고침 버튼(있다면)으로 최신 수치가 반영되며, CSV 내보내기 버튼으로 파일 다운로드가 시작되는 것을 테스트했습니다.

또한, **토큰 입력 UX도 한층 명확해졌습니다.** 토큰이 제대로 설정되지 않았을 때는 주요 기능 버튼들을 **비활성화**하거나, 클릭 시 “토큰이 필요합니다” 경고창을 띄우도록 해서, 사용자가 반드시 토큰을 넣도록 유도했습니다. 토큰 설정 후에는 이러한 제한을 풀어 정상 동작하게 하였습니다. 이러한 UX 향상은 작은 것이지만, 앞으로 **토큰 누락으로 인한 오류를 미연에 방지할 수** 있습니다.

PowerShell 배포 스크립트 주의사항

문제를 재발시키지 않기 위해, **배포 자동화 스크립트(PowerShell)**도 개선하였습니다. 앞서 Cloud Run 섹션에서 언급한 것처럼, 이번 오류의 핵심은 **gcloud 명령의 인자 전달 형식**이 잘못되어 발생했습니다. 이에 대한 일반적인 지침과 본 스크립트에 적용한 개선은 다음과 같습니다:

- **따옴표 사용 최소화:** PowerShell은 Bash와 다르게 문자열 처리 규칙이 복잡합니다. 특히 **명령줄 인자에서 이중 인용부호(")는 제거되기 쉽기 때문에**, 불필요하게 인자를 큰따옴표로 감싸지 않도록 스크립트를 수정했습니다⁴. 예를 들어, 이전에는

```
gcloud run services update-traffic "$serviceName" --to-latest --region "$region"
```

와 같이 작성되었을 수 있는데, 이를

```
gcloud run services update-traffic $serviceName --to-latest --region $region
```

로 변경했습니다. 이렇게 하면 `$serviceName`이나 `$region`에 공백이 없는 한 문제없이 전달됩니다. (만약 공백이 포함된 값이라면 `"$variable"`로 감싸야 하지만, Cloud Run 서비스명이나 리전명은 공백이 없음)

참고: Stack Overflow 사용자들도 PowerShell에서의 인용부호 문제를 자주 겪고 있으며, PowerShell이 외부 프로그램 실행 시 인자를 전달할 때 따옴표를 자동 제거 또는 제대로 escape하지 못하는 버그/특성이 있다고 지적합니다⁵⁹.

- `--region` 등 **인자 명시:** 혹시 스크립트에서 `gcloud config set run/region asia-northeast3` 등을 미리 실행해두고 `--region` 옵션을 생략했다면, 그런 방식보다는 **명령마다** `--`

`region` 과 `--project` 등을 명시적으로 넣는 것을 권장합니다. 명시적으로 넣으면 가독성이 높아지고, 다른 환경에서 실행 시 설정 누락으로 오작동할 위험이 줄어듭니다. 이번 스크립트에도 모든 `gcloud` 명령에 `--region asia-northeast3 --project <프로젝트명>` 을 정확히 넣어주었습니다.

- **오류 처리 및 로그:** 스크립트가 실패했을 때 바로 알 수 있도록 `$ErrorActionPreference = "Stop"` 를 설정하고, `try { ... } catch { ... }` 블록으로 예외를 잡아 **에러 메시지를 명확히 로깅**하도록 했습니다. 예컨대 `update-traffic` 명령 실패 시 `$?` 상태를 체크해 실패면 `Write-Error "Traffic update failed: $($_.Exception.Message)"` 식으로 표시하게 했습니다. 이렇게 하면 나중에 로그만 봐도 어떤 명령이 왜 실패했는지 알 수 있습니다.
- **테스트 및 문서화:** 스크립트 수정 후 동일 환경에서 여러 번 테스트를 거쳐 재현되지 않는 것을 확인했습니다. 그리고 이 내용을 배포 문서에 **Known Issue**로 남겨두어, **PowerShell에서 gcloud 사용 시 인용부호 처리** 주의를 명시했습니다.

이러한 개선으로 배포 **파이프라인의 신뢰성**을 높였으며, 특히 **to-latest 플러그 미적용 문제**는 재발하지 않을 것입니다. 향후 누군가 스크립트를 변경하더라도, 관련 주석과 문서가 있어 같은 실수를 줄일 수 있을 것으로 기대합니다.

QA 자동 점검 버튼 동작 확인 및 개선

QA 자동 점검 기능은 품질 관리 상 중요한 기능으로, 이번에 특히 신경써서 점검했습니다. 앞서 UI 코드 수정사항에서 QA 버튼의 구현을 보완했는데, 여기서는 **올바르게 동작하는지 검증**하고 **추가적인 개선점(폴백 전략)**에 대해 서술합니다.

- **기능 동작 확인:** 수정 후 **QA 자동 점검 버튼**을 클릭하면 프론트엔드에서 `/api/qa/run` 요청을 보내도록 했습니다. 이 때 실제 **네트워크 요청이 발생하는지** 개발자 도구(Network 패널)에서 확인했습니다. 요청 헤더에 **Authorization: Bearer** 토큰이 포함되었고, 응답은 **202 Accepted** (예를 들면 백엔드에서 비동기 작업 큐에 넣고 즉시 응답) 혹은 **200 OK**와 결과 값이 오는 것을 확인했습니다. 백엔드 Cloud Run 로그에서도 해당 엔드포인트에 대한 **요청 수신 로그와 처리 로그**(ex: "QA job started")를 찾아보았습니다. 이를 통해 버튼이 제대로 백엔드를 트리거함을 검증했습니다.
- **결과 표시/폴백:** QA 자동 점검은 아마 **시간이 걸리는 작업**일 것입니다. 따라서 버튼 클릭 후 사용자에게 **즉시 완료되는 작업이 아님**을 인지시킬 필요가 있습니다. 개선된 UI에서는 "QA 자동 점검이 시작되었습니다. 완료까지 몇 분 소요됩니다." 등의 메시지를 alert로 표시했습니다. 추가로, **폴백(fallback)** 시나리오도 고려했습니다. 만약 **백엔드가 응답하지 않거나 오류를 던진 경우**(예: 500 에러), 사용자에게 **"QA 점검 요청 실패: 관리자에게 문의하십시오."** 등의 메시지를 보여주어, 계속 눌러봐도 소용없음을 알 수 있게 했습니다.
- **수동 대안 제공:** 폴백의 또 다른 의미로, **자동 점검이 안 될 경우 대안 경로**를 안내하는 것도 있습니다. 예를 들어, QA 점검 API가 계속 실패한다면, 관리자가 수동으로 검증할 수 있도록 관련 데이터나 로그를 제공하는 기능을 고려할 수 있습니다. 예를 들어 "문제가 발생하면, [로그 다운로드] 버튼을 눌러 로그를 확인하세요" 같은 안내입니다. 현재 시스템에 그런 기능은 없지만, 장기적으로는 **실패 시 로그나 리포트를 제공**하는 것이 품질 관리에 도움이 될 것입니다.
- **주기적 점검 기능:** 한편, QA 자동 점검은 수동 버튼 외에 **주기적인 자동실행 여부**도 점검했습니다. 이번 이슈에서는 관계 없지만, 혹시 스케줄러가 돌아가고 있다면 토큰 문제로 실패했을 수 있어서입니다. 다행히 그런 부분은 없었고, 오로지 **수동 버튼 트리거** 방식으로 구현되어 있었습니다.

QA 기능이 정상 작동함을 확인한 후, **전체 관리 프로세스**를 한번 시연하여 **모든 기능이 유기적으로 동작하는지** 최종 점검했습니다. 토큰 입력 -> KPI 조회 -> QA 실행 -> 보고서 확인의 흐름이 끊김없이 이뤄졌고, 각 단계에서 사용자에게 피드백이 주어지는 것을 확인했습니다.

도메인 캐시 무효화 방법

이번 장애에서 캐시가 크게 한 몫 한 것은 아니지만, 최신 배포 반영 시 간과할 수 없는 부분이므로 **도메인/캐시 관리**에 대한 방법을 정리합니다:

- **브라우저 캐시 무효화:** 관리자는 배포 후 자신의 브라우저에서 **강력 새로고침(캐시 초기화)**을 습관화해야 합니다. Chrome 등에서는 `Ctrl + Shift + R`로 캐시를 무시한 새로고침이 가능합니다. 또한, 배포시 리소스 파일에 해시를 붙이는 등 캐시 갱신 전략을 쓰면 사용자의 브라우저 캐시 문제를 줄일 수 있습니다.
- **Cloud Run CDN 캐시:** Cloud Run 도메인 매핑시 **Cloud CDN**을 옵션으로 활성화할 수 있는데, 만약 이를 썼다면 `gcloud compute url-maps invalidate-cdn-cache` 명령으로 캐시 무효화를 할 수 있습니다. 다행히 이번 서비스는 Cloud CDN을 사용하지 않아 해당 사항은 없었습니다. (Cloud Run 단독 사용 시에는 **동적 콘텐츠는 CDN 캐시되지 않는 것이 기본**입니다 ¹⁰.)
- **CloudFlare 등 프록시 캐시:** `standardai.co.kr`이 CloudFlare 같은 프록시를 통할 경우, 해당 CDN 콘솔에서 **캐시 퍼지(Purge)**를 실행할 수 있습니다. 일반적으로 **정적 파일(JS/CSS)**은 캐싱되고 **HTML/API 응답**은 캐싱 안 하는 설정이 많지만, 혹시 HTML도 캐싱하도록 설정되면 위험하므로 확인이 필요합니다. 우리는 **관리 도구 페이지는 항상 최신이 보여야 하므로**, CloudFlare 사용 시 **Page Rule** 등을 통해 `admin.standardai.co.kr/*` 경로는 **캐시 없음(no cache)**으로 운영하는 편이 안전합니다.
- **DNS 변경 캐시:** 예를 들어 Cloud Run 리비전을 새로 서비스로 만들어 바꿨을 때 CNAME을 변경했다면, 전세계 DNS에 퍼지는 데 시간이 걸립니다. 이럴 때 **TTL(Time-To-Live)** 값을 낮추고 진행하면 캐시 영향을 줄일 수 있습니다. 이번에는 리비전 교체만 있었고 도메인 변경은 없었습니다.

요컨대, **배포 후 캐시 무효화** 작업을 체크리스트에 추가했습니다. 특히 CloudFlare 같은 서비스를 사용할 경우 **배포 자동화에 API 연동하여 새 배포시 캐시 삭제**를 해주면 좋습니다. 이런 부분을 운영 매뉴얼에 명시하여, 배포 담당자가 놓치지 않도록 했습니다.

Definition of Done 체크리스트

마지막으로, 이번 이슈들과 대응 작업을 종합하여 **완료 기준(Definition of Done)**을 정리합니다. 아래 항목들을 모두 충족함을 확인한 후 문제를 완료 처리하였습니다:

- [x] **최신 UI 화면 적용:** 배포한 최신 `index.html` /JS 내용이 실제 관리자 도메인에서 확인된다 (옛 UI 요소가 더 이상 보이지 않음).
- [x] **KPI 수치 갱신:** 유효한 토큰을 입력했을 때 `/api/status`로부터 실제 KPI 값이 수신되어 화면에 표시된다 (더 이상 0 고정치 아님).
- [x] **QA 자동 점검 동작:** “QA 자동 점검” 버튼 클릭 시 백엔드 요청이 성공적으로 수행되고, 사용자가 이에 대한 피드백을 받는다 (예: “시작되었습니다” 메시지 출력). 백엔드 로그상으로도 QA 처리 로직이 실행됨을 확인.
- [x] **보고서/CSV 출력:** “보고서 생성”/“CSV 내보내기” 버튼을 누르면 파일 다운로드 또는 결과 표시가 이뤄진다. 빈 내용이 아닌 실제 데이터가 들어있는 것을 확인.
- [x] **토큰 입력 기능 정상화:** 토큰 입력 후 내부적으로 저장되고, 이후 모든 API 호출에 Authorization 헤더가 포함된다. (개발자 도구 Network 탭에서 각 요청의 Request Headers에 `Authorization: Bearer ...` 존재 확인 ⁸.)
- [x] **프론트엔드 오류 없음:** 브라우저 콘솔에 JavaScript 에러가 발생하지 않는다. 특히 초기 로드 시 fetch 재정의 관련 오류나 기타 ReferenceError 등이 없는 것을 확인.
- [x] **Cloud Run 배포 정상화:** Cloud Run 서비스 `quali-admin-domap`의 **현재 활성 revision**이 최신 코드이며, **트래픽 100% 할당** 상태인 것을 확인 (콘솔 UI 또는 `gcloud run services describe` 결과). 이전 revision들은 더 이상 트래픽을 받지 않음 ⁶.

- [x] **배포 스크립트 오류 해결:** 개선된 PowerShell 배포 스크립트를 통해 **연속 배포 테스트**를 진행하여 오류 없이 새 revision 배포 및 즉시 트래픽 전환이 이루어지는지 확인. (실제 환경에 적용하여 2~3회 배포, 문제 재발 없음)
- [x] **캐시 이슈 해결:** 캐시 정책 적용 후, 일반 새로고침으로도 항상 최신 콘텐츠가 보이는지 확인. 필요시 CloudFlare 캐시 등도 모두 퍼지 완료.
- [x] **문서 및 전파:** 본 보고서의 내용을 관련 개발자, 운영자에게 공유하고, **향후 유사 문제 발생 시의 대처법**을 숙지시켰음. 특히 **토큰 누락, 이벤트 바인딩 누락** 등에 대한 체크리스트를 QA 프로세스에 포함.

以上的 모든 항목이 충족되어, QualiJournal 관리자 시스템의 최근 문제들은 **완전히 해결**되었음을 선언합니다. 앞으로는 배포 전후로 위 체크리스트를 참고하여 한층 견고한 운영을 유지하고자 합니다. 각 팀원분들도 관련 내용을 숙지하셔서, 유사한 이슈 발생을 미연에 방지해 주시기 바랍니다.

<!-- Sources --> 참조: 1 2 6 3 8 4

1 Traffic Routing in Cloud Run. Cloud Run is Google's Fully Managed... | by Sushil Kumar | Google Cloud - Community | Medium

<https://medium.com/google-cloud/traffic-routing-in-cloud-run-23090b6873bd>

2 reactjs - Cloud Run Caching old revisions - Stack Overflow

<https://stackoverflow.com/questions/75438961/cloud-run-caching-old-revisions>

3 ajax - Intercept fetch() API requests and responses in JavaScript - Stack Overflow

<https://stackoverflow.com/questions/45425169/intercept-fetch-api-requests-and-responses-in-javascript>

4 5 9 windows - PowerShell stripping double quotes from command line arguments - Stack Overflow

<https://stackoverflow.com/questions/6714165/powershell-stripping-double-quotes-from-command-line-arguments>

6 Rollbacks, gradual rollouts, and traffic migration | Cloud Run | Google Cloud

<https://cloud.google.com/run/docs/rollouts-rollbacks-traffic-migration>

7 10 Firebase hosting caches Google Cloud Run requests - Stack Overflow

<https://stackoverflow.com/questions/58729871/firebase-hosting-caches-google-cloud-run-requests>

8 How to send Bearer Token with JavaScript Fetch API?

<https://reqbin.com/code/javascript/ricgaie0/javascript-fetch-bearer-token>